1. Introduction & Problem Statement

Data-layout transformations are critical in hardware accelerators for deep neural networks (DNNs). High-performance execution units—such as systolic arrays, vector processing units, and line buffers—require multi-dimensional feature maps to be flattened into linear, contiguous streams during memory transfers and processing element (PE) feeding.

The objective of this assignment is to implement and evaluate data-layout transformations between a 4D image/feature-map tensor $I \in \mathrm{R}^{B \times C \times H \times W}$ and a 2D flattened representation $F \in \mathrm{R}^{B \times (C \cdot H \cdot W)}$:

1. **Flattening:** $B \times C \times H \times W \longrightarrow B \times (C \cdot H \cdot W)$
2. **Reconstruction:** $B \times (C \cdot H \cdot W) \longrightarrow B \times C \times H \times W$

As required by the assignment rules, these transformations were implemented strictly using explicit nested loops and index arithmetic without leveraging library shape functions (such as `reshape`, `view`, `flatten`, or `ravel`).

---

2. Algorithm Pseudocode

2.1 Flattening Algorithm ($B \times C \times H \times W \rightarrow B \times (C \cdot H \cdot W)$)

For a tensor in row-major $BCHW$ order, the spatial-channel multi-index (c, h, w) maps to a 1D linear index k via row-major index accumulation:

$$\mathrm{Stride}_c = H \cdot W, \quad \mathrm{Stride}_h = W, \quad \mathrm{Stride}_w = 1$$

```
Input: Tensor I of shape (B, C, H, W)
Output: Matrix F of shape (B, C * H * W)

Initialize F[B, C * H * W] with zeros
FOR b = 0 TO B - 1 DO
    FOR c = 0 TO C - 1 DO
        FOR h = 0 TO H - 1 DO
            FOR w = 0 TO W - 1 DO
                k = c * (H * W) + h * W + w
                F[b, k] = I[b, c, h, w]
            END FOR
        END FOR
    END FOR
END FOR
```

```
RETURN F
```

2.2 Reconstruction Algorithm ($B \times (C \cdot H \cdot W) \rightarrow B \times C \times H \times W$)

Reconstruction decomposes the 1D index $k \in [0, C \cdot H \cdot W - 1]$ back into individual multi-dimensional coordinates (c, h, w) using floor division and modulo arithmetic:

```
Input: Matrix F of shape (B, C * H * W), Target shape (B, C, H, W)
Output: Reconstructed Tensor I_hat of shape (B, C, H, W)

HW = H * W
Initialize I_hat[B, C, H, W] with zeros
FOR b = 0 TO B - 1 DO
    FOR k = 0 TO (C * HW) - 1 DO
        c = k // HW             // Extract channel index
        r = k % HW              // Remaining spatial offset
        h = r // W              // Extract row height index
        w = r % W              // Extract column width index
        I_hat[b, c, h, w] = F[b, k]
    END FOR
END FOR
RETURN I_hat
```

---

3. Manual Indexing Example

To demonstrate the mapping explicitly, consider a small tensor I with dimensions $B = 1$, $C = 2, H = 2, W = 3$:

- Spatial stride ($H \cdot W$) = $2 \times 3 = 6$
- Total element count per batch item ($C \cdot H \cdot W$) = $2 \times 6 = 12$

Input Tensor Layout:

$$\text{Channel 0 } (c = 0): \begin{bmatrix} 1.0 & 2.0 & 3.0 \\ 4.0 & 5.0 & 6.0 \end{bmatrix}, \quad \text{Channel 1 } (c = 1): \begin{bmatrix} 7.0 & 8.0 & 9.0 \\ 10.0 & 11.0 & 12.0 \end{bmatrix}$$

Coordinate Mapping Table:

				Address		

Batch (b)	Channel (c)	Height (h)	Width (w)	Address Arithmetic: $k = c(HW) + h(W) + w$	Linear Index (k)	Tensor Value
0	0	0	0	$0(6) + 0(3) + 0$	**0**	**1.0**
0	0	0	1	$0(6) + 0(3) + 1$	**1**	**2.0**
0	0	0	2	$0(6) + 0(3) + 2$	**2**	**3.0**
0	0	1	0	$0(6) + 1(3) + 0$	**3**	**4.0**
0	0	1	1	$0(6) + 1(3) + 1$	**4**	**5.0**
0	0	1	2	$0(6) + 1(3) + 2$	**5**	**6.0**
0	1	0	0	$1(6) + 0(3) + 0$	**6**	**7.0**
0	1	0	1	$1(6) + 0(3) + 1$	**7**	**8.0**
0	1	0	2	$1(6) + 0(3) + 2$	**8**	**9.0**
0	1	1	0	$1(6) + 1(3) + 0$	**9**	**10.0**
0	1	1	1	$1(6) + 1(3) + 1$	**10**	**11.0**
0	1	1	2	$1(6) + 1(3) + 2$	**11**	**12.0**

---

4. Experimental Setup & Results

Experiments were conducted across standard image datasets (MNIST, CIFAR-10) as well as synthetic multi-channel feature maps representing modern DNN layer channels ($C \in \{1, 3, 8, 16, 32, 64, 128, 256, 500\}$).

The reconstruction fidelity was verified using two error metrics:

1. **Maximum Absolute Error (E_{max}):**

$$E_{max} = \max_{b,c,h,w} |I[b, c, h, w] - \hat{I}[b, c, h, w]|$$

2. **Mean Absolute Error (MAE):**

$$\text{MAE} = \frac{1}{BCHW} \sum_{b,c,h,w} |I[b, c, h, w] - \hat{I}[b, c, h, w]|$$

Table 1: Experimental Results Across Various Tensor Configurations

Dataset / Input	B	C	$H \times W$	E_{max}	MAE
MNIST / sample					

| 1

| 1

| 28×28 | 0.00×10^{0} | 0.00×10^{0} | | CIFAR / sample

| 1

| 3

| 32×32 | 0.00×10^{0} | 0.00×10^{0} | | Synthetic fmap1

| 2 | 8 | 14×14 | 0.00×10^{0} | 0.00×10^{0} | | Synthetic fmap2

| 2 | 16

| 14×14 | 0.00×10^{0} | 0.00×10^{0} | | Synthetic fmap3 | 2 | 32 | 14×14 | 0.00×10^{0} | 0.00×10^{0} | | Synthetic fmap4

| 2 | 64

| 14×14 | 0.00×10^{0} | 0.00×10^{0} | | Synthetic fmap5 | 2 | 128

| 14×14 | 0.00×10^{0} | 0.00×10^{0} | | Synthetic fmap6 | 2 | 256 | 7×7 | 0.00×10^{0} | 0.00×10^{0} | | Synthetic fmap7 | 2 | 500

| 7×7 | 0.00×10^{0} | 0.00×10^{0} |
